SE3002 - Software Quality Engineering: Assignment 2

Student: Afraz Khan | Roll number: 23p3034 | GitHub: theafrazkhan

Submission prepared: 11 November 2026 (Asia/Karachi).

1. Scope and method

I tested the supplied Student Course Registration System against REQUIREMENTS.md. The original SRS DOCX was also read and agrees with the Markdown requirements. The application was not redesigned. Black-box cases were designed and committed before inspecting implementation logic. Initial failures were saved before fixes. Then I inspected the source, added structural tests, and applied five small fixes. The work was prepared with coding-assistant help; the named student should verify and understand the work before submitting it.

2. Black-box design

The test inventory contains 21 ECP rows, 25 BVA rows, 11 decision-table rows and 11 state rows. These are 68 technique-to-case mappings, consolidated to 53 unique black-box executions by removing 15 exact repeated mappings.

ECP covers valid and invalid credits, zero/positive/negative dues, empty/all/ partial/no prerequisites, unrelated completed courses, and available/full seats. BVA covers current credits -1,0,1,17,18,19; course credits 0,1,2,3,4,5; combined totals 17,18,19; capacity 0,1,2; enrollment -1,0,capacity-1,capacity, capacity+1; and dues -1,0,1. Cases use pytest.raises for specified validation errors.

The reduced decision table has two validation rules and six status rules. D09-D12 exercise competing conditions in the specified order: dues, duplicate, prerequisites, credits, availability. D19 validates dues before capacity; D20 checks capacity validation before positive-dues blocking. D01 and D08 cover available and full seats; D22 allows the exact 18-credit total.

Every decision case checks object state. S01-S11 check successful registration, first/repeated waitlisting, each rejection, validation failure, and both withdrawal outcomes with complete object snapshots. Each case uses fresh objects.

Full representative inputs, expected outputs, requirement IDs and test IDs are in blackbox_test_design.xlsx, decision_table.xlsx and cases.json.

3. Optimization and traceability

D01, D02 and D08 each serve ECP, BVA and decision-table design with one execution. C02-C06, C09, C10 and C14 each serve ECP and BVA, as does D04. The spreadsheets retain each technique mapping. Identical outputs alone were not used to merge tests: operation, complete fixture, numeric inputs, oracle and state assertions must all match. D08 and S02 remain separate because one observes a decision and the other applies a state change. Boundary cases with different inputs remain separate even when both return True or the same registration status.

4. Measured execution and coverage

Phase	Passed	Failed	Statement coverage	Branch coverage	Missing statements
initial-blackbox	39	14	54/57 (94.74%)	23/26 (88.46%)	[51, 59, 71]
pre-fix-combined	49	14	56/57 (98.25%)	25/26 (96.15%)	[59]
final-regression	63	0	58/58 (100.00%)	26/26 (100.00%)	[]

The first two phases use the same unchanged starter source. White-box additions improve coverage without correcting any defect. Coverage is measured for the whole registration module using pytest-cov with branch measurement enabled. The final module has one additional statement: the missing enrollment decrement. JSON, JUnit, commands, versions, source commits, source hashes and console logs are preserved under evidence/.

The coverage tool's combined Cover percentage is not used as the statement percentage. Original missing statements are 51,59,71, with branch gaps 50->51,58->59,70->71. Pre-fix combined coverage leaves only line 59 and edge 58->59 missing; the final regression has no gaps.

5. Confirmed defects and fixes

BUG-01: Exactly 18 credits is incorrectly rejected

Requirement: FR-01. Reproduce: Call check_credit_limit(15, 3).

Expected: True. Actual in the starter: False.

Failed test: C13. Fix: Change the total comparison from <18 to <=18. Regression IDs: C08, C13, D22. All now pass.

Issue: https://github.com/theafrazkhan/SQE-Assignment-2/issues/1. Status: corrected and regression-tested; see GitHub issue/PR state for merge status.

BUG-02: Only some prerequisites incorrectly allow registration

Requirement: FR-03/06. Reproduce: Call prerequisites_satisfied({'CS101'}, {'CS101', 'MATH101'}).

Expected: False. Actual in the starter: True.

Failed test: P03. Fix: Check required.issubset(completed). Regression IDs: P03, D21. All now pass.

Issue: https://github.com/theafrazkhan/SQE-Assignment-2/issues/2. Status: corrected and regression-tested; see GitHub issue/PR state for merge status.

BUG-03: Positive dues up to Rs. 1000 do not block registration

Requirement: FR-04/09. Reproduce: Use default decision fixture (docs/cases.json) and set outstanding_dues=1. Call registration_decision.

Expected: DUES_BLOCK. Actual in the starter: REGISTERED.

Failed test: D02. Fix: Change the dues threshold from >1000 to >0. Regression IDs: D02, D03, D09, S04. All now pass.

Issue: https://github.com/theafrazkhan/SQE-Assignment-2/issues/3. Status: corrected and regression-tested; see GitHub issue/PR state for merge status.

BUG-04: A full course is registered instead of waitlisted

Requirement: FR-08/12. Reproduce: Use default decision fixture; set capacity=2 and enrolled=2. Call registration_decision. Calling register_course also incorrectly raises enrollment to 3.

Expected: WAITLISTED. Actual in the starter: REGISTERED.

Failed test: D08. Fix: Use enrolled == capacity; validation already excludes over-capacity states. Regression IDs: D08, D15, S02, S03. All now pass.

Issue: https://github.com/theafrazkhan/SQE-Assignment-2/issues/4. Status: corrected and regression-tested; see GitHub issue/PR state for merge status.

BUG-05: Withdrawal does not reduce course enrollment

Requirement: FR-13. Reproduce: Student has CS201 registered, 6 credits; CS201 has 3 credits, capacity=2 and enrolled=1. Call withdraw_course.

Expected: True; course removed, credits=3, enrolled=0. Actual in the starter: True; course removed, credits=3, enrolled remains 1.

Failed test: S10. Fix: Subtract one from course.enrolled on successful withdrawal. Regression IDs: S10. All now pass.

Issue: https://github.com/theafrazkhan/SQE-Assignment-2/issues/5. Status: corrected and regression-tested; see GitHub issue/PR state for merge status.

6. CFG, paths and manual condition analysis

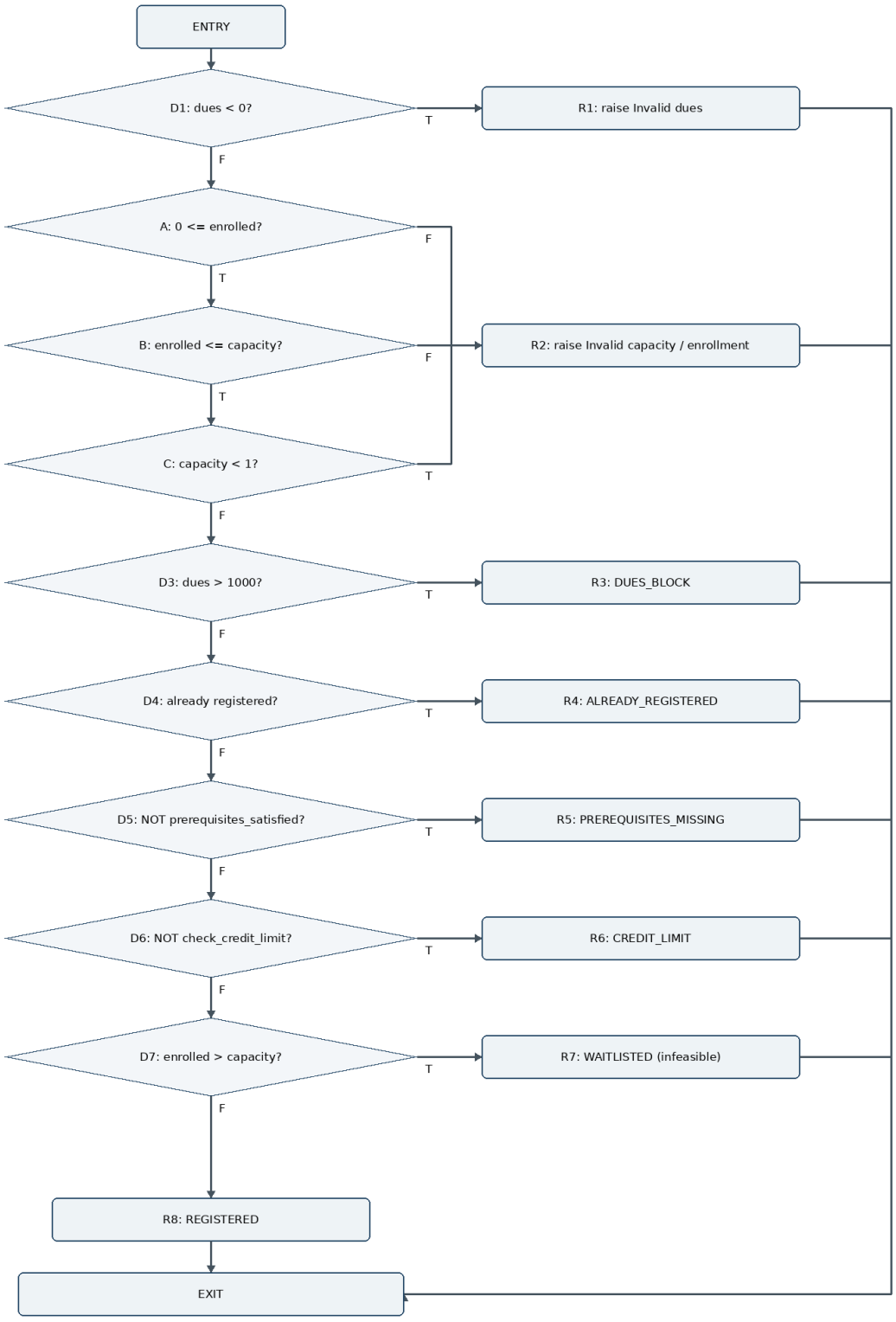
The following analysis uses original starter line numbers, retained in evidence/starter_registration.py. The included CFG is the original function.

Line references below refer to the preserved starter in evidence/starter_registration.py. Black-box design and its initial run were completed before source inspection.

CFG and complexity

registration_decision() - original starter CFG

Expanded short-circuit comparisons; explicit returns/raises share one exit.



N=19, E=27: V(G)=27-19+2=10; 9 predicate nodes + 1 = 10.

See cfg.png. Rectangles are return/raise nodes (processing); diamonds are predicate nodes. ENTRY and EXIT are the two control nodes. All explicit returns and raises connect to one synthetic EXIT. Calls to the two helpers are atomic processing within their predicate: their internal decisions and implicit exception edges are not expanded into this intraprocedural graph. W02/W03 separately test propagation.

At source-if level there are seven predicates (lines 46,48,50,52,54,56,58). With entry, seven decisions, eight terminal actions and exit: N=17, E=23. V(G)=23-17+2=8, matching 7+1.

For the explicit short-circuit convention in cfg.png, line 48 expands into: A = 0 <= enrolled; B = enrolled <= capacity; C = capacity < 1. The decision is NOT(A AND B) OR C. B is skipped when A is false; C is skipped when A or B is false. This replaces one predicate with three: N=19, E=27. V(G)=27-19+2=10, matching 9+1. Both counts describe the same function at different granularities. pytest-cov counts source branches, not these atomic edges.

Independent basis paths

Use the expanded graph. Prefix ENTRY and suffix EXIT are implicit below. Each path adds a distinct terminal edge or a new validation-failure edge. Paths are structural; the last path cannot execute with the starter's guards.

Path	Predicate route and terminal	Representative case
P1	D1 F,A T,B T,C F,D3 F,D4 F,D5 F,D6 F,D7 F -> R8	D01 / W07
P2	D1 T -> R1	D04
P3	D1 F,A F -> R2	D16 / W08
P4	D1 F,A T,B F -> R2	D18 / W09
P5	D1 F,A T,B T,C T -> R2	D13 / W10
P6	D1 F,A T,B T,C F,D3 T -> R3	W01
P7	valid prefix,D3 F,D4 T -> R4	D05
P8	valid prefix,D3 F,D4 F,D5 T -> R5	D06
P9	valid prefix,D3 F,D4 F,D5 F,D6 T -> R6	D07
P10	valid prefix,D3 F,D4 F,D5 F,D6 F,D7 T -> R7	Infeasible before fix; D08 after fix

Line 48 requires enrolled <= capacity to continue, while line 58 requires enrolled > capacity for WAITLISTED. Therefore P10 is contradictory, even for invalid inputs: validation raises before reaching it. No fabricated test or coverage exclusion is used. After the equality fix, D08 executes this path.

Structural additions and coverage gaps

W01 adds the dues >1000 true edge and line 51. W02/W03 verify exceptions propagating from the credit helper without mutation. W04-W06 isolate register_course by stubbing only registration_decision's returned status: W04 executes the previously unreachable line 71; W05 exercises an existing ID; W06 tests the false first operand and skipped membership check. These are unit tests of the application layer, not evidence that the original registration_decision can return WAITLISTED. W07-W10 target each short-circuit validation outcome. Some overlap with black-box cases is intentional structural traceability; the black-box technique optimization does not remove these additions.

The initial missing statements are 51,59,71; missing branches are 50->51,58->59,70->71. With white-box additions before fixes, only 59 and 58->59 remain missing. The final fixed module has no uncovered statements or source branches. Detailed totals are in testing_report.md and evidence JSON. Coverage percentages are measured separately as covered statements/total statements and covered branches/total branches; the terminal's Cover column combines the two and must not be mislabeled as statement coverage.

Atomic conditions in registration_decision

Atom	Meaning in original function	True case	False case
D1	dues < 0	D04	D01

Atom	Meaning in original function	True case	False case
A	0 <= enrolled	W07	W08
B	enrolled <= capacity (only when A is true)	W07	W09
C	capacity < 1 (only when A and B are true)	W10	W07
D3	dues > 1000	W01	D01
D4	code in registered_courses	D05	D01
D5	not prerequisites_satisfied(...)	D06	D01
D6	not check_credit_limit(...)	D07	D01
D7	enrolled > capacity	Infeasible	D01

Scope of the manual condition percentage: these nine atomic predicates in the requested function. Helper internals are outside this scope; the additional register_course compound decision is analyzed separately below. A returned helper result is one atom at this function's level. Initial black-box: 16 of 18 evaluated True/False outcomes = 88.89% (D3 true and D7 true missing). Pre-fix combined: 17/18 = 94.44%, D7 true infeasible. Fixed: 18/18 = 100% with D3 now dues >0 and D7 now enrolled == capacity. These are manual condition results; pytest-cov does not measure them.

Short-circuit truth table and MC/DC for validation

NE means Python did not evaluate that operand, and is not counted as True/False. D is NOT(A AND B) OR C, where True raises the validation exception.

Case	enrolled	capacity	A	B	C	D
W07	0	1	T	T	F	F
W08	-1	1	F	NE	NE	T
W09	2	1	T	F	NE	T
W10	0	0	T	T	T	T

A independent effect: W07/W08. In W08, latent B is T (-1<=1) and latent C is F (1<1), matching W07; both are masked by A being false. B independent effect: W07/W09. A stays T; latent C stays F (capacity=1). C independent effect: W07/W10. A and B stay T; C changes F->T. All three pairs change D. Thus 3/3 atomic conditions have a masking MC/DC pair (100%) for this compound decision. This explicitly allows operands skipped by short-circuiting. Strict all-operands-evaluated unique-cause MC/DC is not claimed: Python necessarily skips B/C for relevant invalid rows. Each atom is actually observed both T and F over the four tests: 6/6 condition outcomes, 100%. Changing input values is allowed as long as the other atomic truth values remain fixed or masked as stated.

AND decision in register_course (line 70)

A = decision == WAITLISTED; B = student ID not in waitlist; D = A AND B. The table is reached only when the preceding REGISTERED condition is false.

Case	Returned status	Waitlist before	A	B	D
W04	WAITLISTED	[]	T	T	T
W05	WAITLISTED	[S001]	T	F	F
W06	DUES_BLOCK	[]	F	NE	F

W04/W06 show A's independent effect with latent B=T fixed. W04/W05 show B's independent effect with A=T fixed. Masking MC/DC is 2/2=100%; evaluated condition outcomes are 4/4=100%. Before fixes these rows use the documented stub; after fixes the black-box S02/S03/S04 cases exercise the corresponding end-to-end behaviors. No manual percentage here is inferred from branch coverage.

7. GitHub collaboration and CI evidence

Repository: https://github.com/theafrazkhan/SQE-Assignment-2 (private).

• blackbox-testing (base at creation: main): https://github.com/theafrazkhan/SQE-Assignment-2/pull/6

• whitebox-testing (base at creation: blackbox-testing): https://github.com/theafrazkhan/SQE-Assignment-2/pull/7

• defect-fixes (base at creation: whitebox-testing): https://github.com/theafrazkhan/SQE-Assignment-2/pull/8

The Actions workflow runs on push and PR, installs requirements.txt, runs pytest with branch coverage, and uploads JUnit/XML/HTML artifacts even if tests fail. No continue-on-error, xfail or skipped defects are used to hide the initial bugs. Actual GitHub run links and conclusions are recorded below and in github_runs.json.

• defect-fixes: success - https://github.com/theafrazkhan/SQE-Assignment-2/actions/runs/43078017623

• whitebox-testing: failure - https://github.com/theafrazkhan/SQE-Assignment-2/actions/runs/43078012356

• blackbox-testing: failure - https://github.com/theafrazkhan/SQE-Assignment-2/actions/runs/43078006912

• main: failure - https://github.com/theafrazkhan/SQE-Assignment-2/actions/runs/43077947677

8. Contributions, review status and remaining action

Afraz Khan (23p3034, theafrazkhan) is the sole named student. The repository contains requirements-based test design, test implementation, structural analysis, defect reports/fixes, CI setup and reporting prepared for this solo submission with coding-assistant help. No multi-member contribution is claimed.

Independent peer review is PENDING, as explicitly agreed for this solo work. There is no claim that another member reviewed or approved these PRs, or that peer feedback was addressed. The instructor must confirm how the C4 criterion applies to solo work. This is the remaining collaboration requirement; automated tests and a self-check cannot substitute for another member's review.

The repository is private. Give the instructor access before submission. Review review_notes.md for viva topics and explain each change in your own words.

9. Reproduction and practical limits

Install requirements.txt and run python -m pytest tests/ --cov=registration --cov-branch --cov-report=term-missing. The exact saved commands and environment versions are in each evidence phase's run.json. Use tools/run_evidence.py with a new phase name for future runs. The original source is in the starter commit and the preserved evidence copy; the initial branches reproduce the genuine failures.

Tests follow the SRS numeric scope. They do not assert behavior for identifiers, wrong collection types, persistence, concurrency or payment processing, which are outside scope. High source coverage does not prove correctness: the starter already had 94.74% statement coverage while failing 14 requirements-based cases. The isolated waitlist cases and masking MC/DC conventions are disclosed above.